



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего
образования
"МИРЭА — Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №6
по дисциплине
«Разработка баз данных»

Выполнил студент группы ИНБО-20-23

Боргачев Т. М.

Принял доцент кафедры ЦТ

Исаева М.В.

Москва 2025

ПРАКТИЧЕСКАЯ РАБОТА №6. ТРИГГЕРЫ И КУРСОРЫ В POSTGRESQL

Цель: формирование углубленных практических навыков по управлению данными и реализации сложной бизнес-логики в СУБД PostgreSQL с использованием триггеров и курсоров.

Постановка задачи:

Задание №1: триггеры.

Проанализировать предметную область своей базы данных и выявить не менее трёх бизнес-правил, реализация которых в виде ограничений целостности возможна только с помощью триггеров. ВАЖНО: триггеры должны быть разными.

Для каждого правила:

- описать алгоритм его работы, указав таблицу, событие и последовательность действий;
- написать код триггерной функции на PL/pgSQL и оператор CREATE TRIGGER;
- продемонстрировать работу триггера на примерах DML-операций, которые как успешно выполняются, так и корректно прерываются триггером (два запроса).

Задание №2: курсоры.

Разработать два скрипта на PL/pgSQL, демонстрирующих оба способа обработки данных:

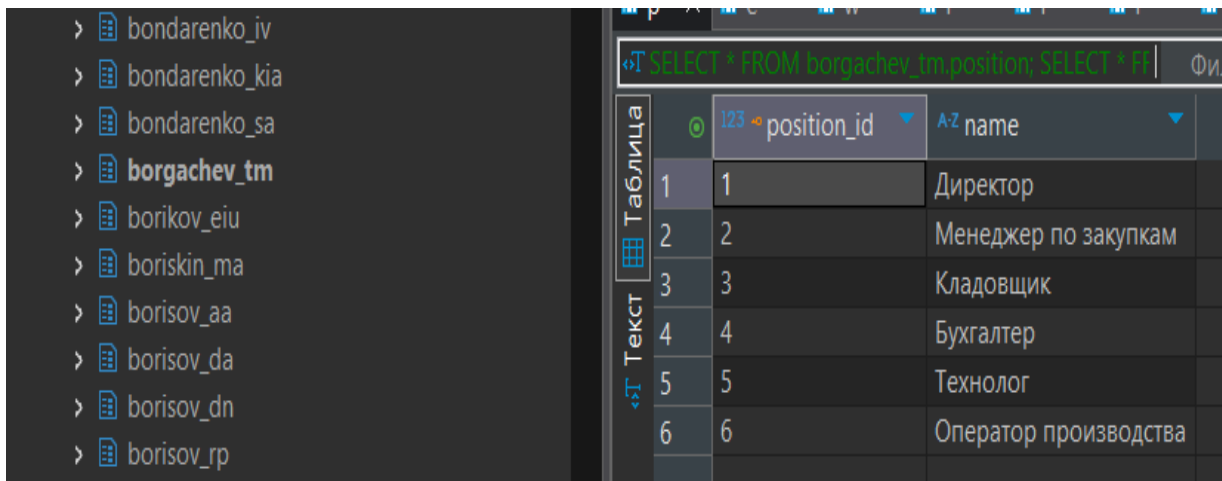
1. С использованием явного курсора (DECLARE/OPEN/FETCH/CLOSE).
2. С использованием неявного курсора (цикл FOR...IN).

Каждый SQL-запрос сопровождать комментарием, объясняющим его назначение и логику работы с учетом специфики вашей базы данных.

ХОД РАБОТЫ

Задание №1: триггеры.

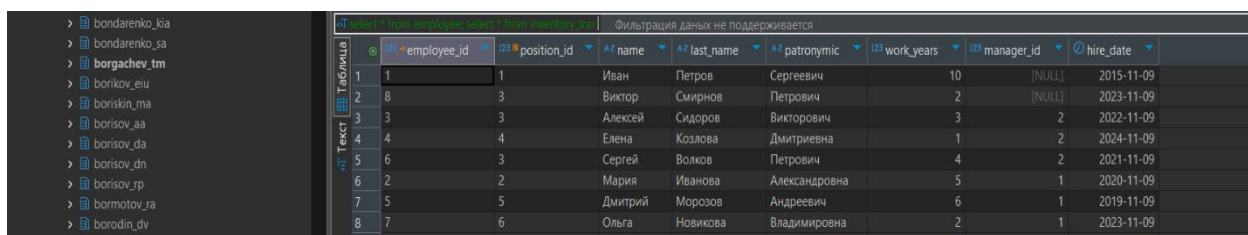
Таблица position представлена на Рисунке 1.



position_id	name
1	Директор
2	Менеджер по закупкам
3	Кладовщик
4	Бухгалтер
5	Технолог
6	Оператор производства

Рисунок 1 —Таблица position

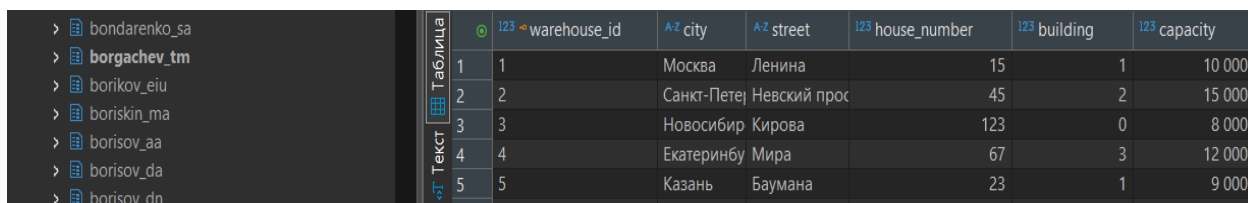
Таблица employee представлена на Рисунке 2.



employee_id	position_id	name	last_name	patronymic	work_years	manager_id	hire_date
1	1	Иван	Петров	Сергеевич	10	[NULL]	2015-11-09
2	8	Виктор	Смирнов	Петрович	2	[NULL]	2023-11-09
3	3	Алексей	Сидоров	Викторovich	3	2	2022-11-09
4	4	Елена	Козлова	Дмитриевна	1	2	2024-11-09
5	6	Сергей	Волков	Петрович	4	2	2021-11-09
6	2	Мария	Иванова	Александровна	5	1	2020-11-09
7	5	Дмитрий	Морозов	Андреевич	6	1	2019-11-09
8	7	Ольга	Новикова	Владимировна	2	1	2023-11-09

Рисунок 2 —Таблица employee

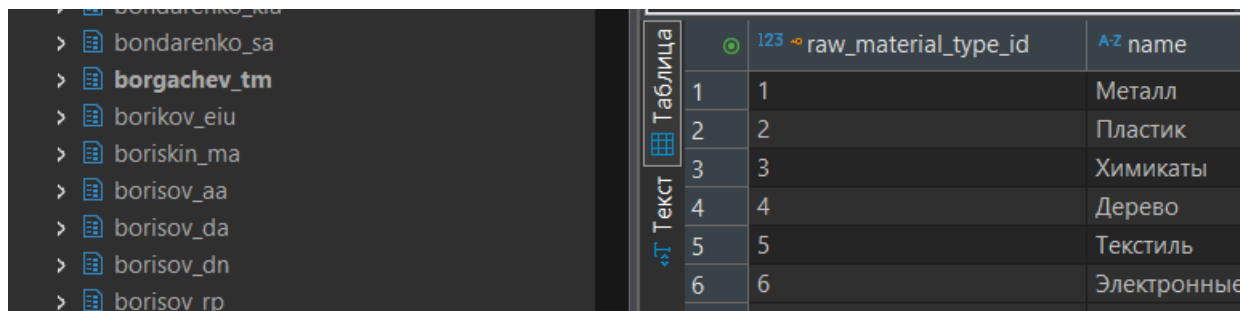
Таблица warehouse представлена на Рисунке 3.



warehouse_id	city	street	house_number	building	capacity
1	Москва	Ленина	15	1	10 000
2	Санкт-Петербург	Невский прот	45	2	15 000
3	Новосибирск	Кирова	123	0	8 000
4	Екатеринбург	Мира	67	3	12 000
5	Казань	Баумана	23	1	9 000

Рисунок 3 —Таблица warehouse

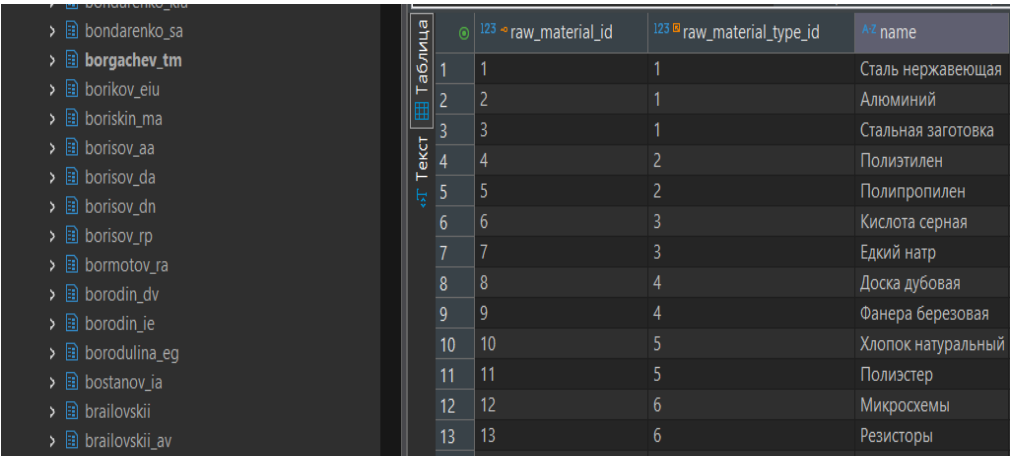
Таблица raw_material_type представлена на Рисунке 4.



raw_material_type_id	name
1	Металл
2	Пластик
3	Химикаты
4	Дерево
5	Текстиль
6	Электронные

Рисунок 4 —Таблица raw_material_type

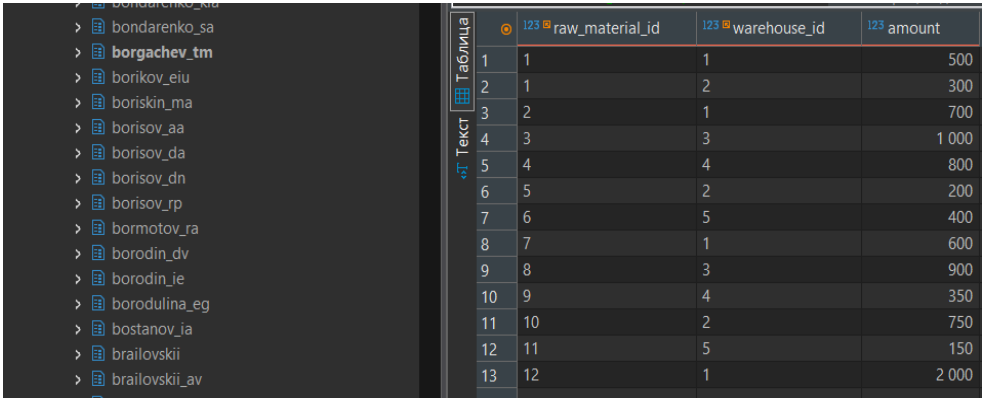
Таблица raw_material представлена на Рисунке 5.



	123 raw_material_id	123 raw_material_type_id	A-Z name
1	1	1	Сталь нержавеющая
2	2	1	Алюминий
3	3	1	Стальная заготовка
4	4	2	Полиэтилен
5	5	2	Полипропилен
6	6	3	Кислота серная
7	7	3	Едкий натр
8	8	4	Доска дубовая
9	9	4	Фанера березовая
10	10	5	Хлопок натуральный
11	11	5	Полиэстер
12	12	6	Микросхемы
13	13	6	Резисторы

Рисунок 5 —Таблица raw_material

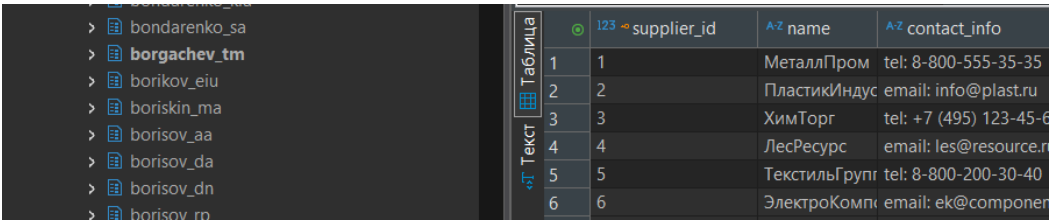
Таблица inventory представлена на Рисунке 6.



	123 raw_material_id	123 warehouse_id	123 amount
1	1	1	500
2	1	2	300
3	2	1	700
4	3	3	1 000
5	4	4	800
6	5	2	200
7	6	5	400
8	7	1	600
9	8	3	900
10	9	4	350
11	10	2	750
12	11	5	150
13	12	1	2 000

Рисунок 6 —Таблица inventory

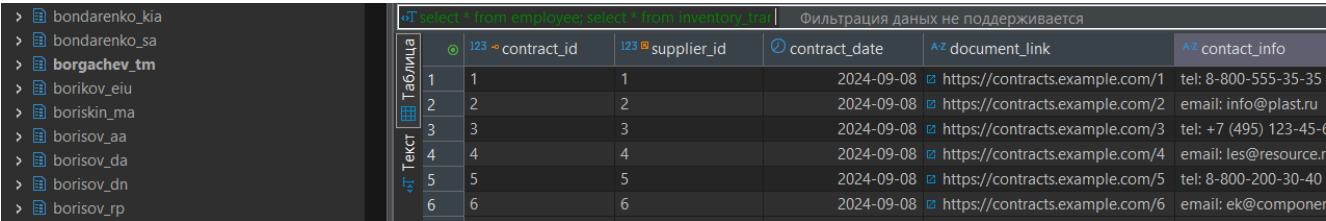
Таблица supplier представлена на Рисунке 7.



	123 supplier_id	A-Z name	A-Z contact_info
1	1	МеталлПром	tel: 8-800-555-35-35
2	2	ПластикИндус	email: info@plast.ru
3	3	ХимТорг	tel: +7 (495) 123-45-6
4	4	ЛесРесурс	email: les@resource.ru
5	5	ТекстильГрупп	tel: 8-800-200-30-40
6	6	ЭлектроКомп	email: ek@componen

Рисунок 7 —Таблица supplier

Таблица contract представлена на Рисунке 8.



	123 contract_id	123 supplier_id	contract_date	A-Z document_link	A-Z contact_info
1	1	1	2024-09-08	https://contracts.example.com/1	tel: 8-800-555-35-35
2	2	2	2024-09-08	https://contracts.example.com/2	email: info@plast.ru
3	3	3	2024-09-08	https://contracts.example.com/3	tel: +7 (495) 123-45-6
4	4	4	2024-09-08	https://contracts.example.com/4	email: les@resource.ru
5	5	5	2024-09-08	https://contracts.example.com/5	tel: 8-800-200-30-40
6	6	6	2024-09-08	https://contracts.example.com/6	email: ek@componen

Рисунок 8 —Таблица contract

Таблица delivery_act представлена на Рисунке 9.

	act_id	warehouse_id	delivery_date
1	1	1	2025-09-03
2	2	2	2025-09-04
3	3	3	2025-09-05
4	4	4	2025-09-06
5	5	5	2025-09-07

Рисунок 9 —Таблица delivery_act

Таблица material_order представлена на Рисунке 10.

	order_id	raw_material_id	employee_id	supplier_id	contract_id	order_date
1	1	1	3	1	1	2025-09-03 1
2	2	2	3	1	1	2025-09-03 1
3	3	1	3	1	1	2025-09-03 1
4	4	2	3	1	1	2025-09-03 1
5	5	1	3	1	1	2025-09-03 1
6	6	2	3	1	1	2025-09-03 1
7	7	1	3	1	1	2025-09-07 5
8	8	1	3	1	1	2025-09-07 5
9	9	1	3	1	1	2025-09-07 5
10	10	1	3	1	1	2025-09-07 5
11	11	1	3	1	1	2025-09-07 5
12	34	1	3	1	1	2024-01-15 1
13	35	2	3	2	2	2024-01-20 2
14	36	3	2	3	3	2024-02-05 3
15	37	4	3	4	4	2024-02-10 4
16	38	5	2	5	5	2024-02-15 5
17	39	6	3	1	1	2024-03-01 1
18	40	7	2	2	2	2024-03-05 2
19	41	8	3	3	3	2024-03-10 3
20	42	9	2	4	4	2024-03-15 4
21	43	10	3	5	5	2024-04-01 5
22	44	11	2	1	1	2024-04-05 1
23	45	12	3	2	2	2024-04-10 2

Рисунок 10 —Таблица material_order

Таблица detail_type представлена на Рисунке 11.

	detail_type_id	name
1	1	Металл
2	2	Пластик
3	3	Химикаты
4	4	Дерево
5	5	Текстиль
6	6	Электронные
7	7	Турбина
8	8	Сталь

Рисунок 11 —Таблица detail_type

Таблица detail представлена на Рисунке 12.

	detail_id	detail_type_id	name
1	1	1	Деталь 1
2	2	2	Деталь 2
3	3	7	Турбина ГТД
4	4	8	Стальная деталь

Рисунок 12 —Таблица detail

Таблица detail_material представлена на Рисунке 13.

	123 detail_id	123 raw_material_id	123 amount
1	4	1	10,5
2	1	1	5,2
3	1	2	8,7
4	2	4	12,3

Рисунок 13 — Таблица detail_material

Таблица inventory_transaction представлена на Рисунке 14.

	123 transaction_id	123 inventory_id	123 employee_id	123 detail_id	123 amount_spent	transaction_date
1	1	1	3	4	50	2025-10-27 10:00:17.465
2	2	2	3	1	30	2025-10-27 10:00:17.465
3	3	5	6	2	25,5	2025-10-27 10:00:17.465
4	4	7	3	[NULL]	10	2025-10-27 10:00:17.465

Рисунок 14 — Таблица inventory_transaction

Таблица inventory_transaction_audit представлена на Рисунке 15.

	123 audit_id	123 transaction_id	123 inventory_id	123 employee_id	123 detail_id	123 amount_spent	transaction_date	operation	audit_timestamp
1	1	1	1	3	4	50	2025-10-27 10:00:17.465	HISTORICAL	2025-11-09 16:45:28.556
2	2	2	2	3	1	30	2025-10-27 10:00:17.465	HISTORICAL	2025-11-09 16:45:28.556
3	3	3	5	6	2	25,5	2025-10-27 10:00:17.465	HISTORICAL	2025-11-09 16:45:28.556
4	4	4	7	3	[NULL]	10	2025-10-27 10:00:17.465	HISTORICAL	2025-11-09 16:45:28.556
5	5	5	1	[NULL]	[NULL]	100	2025-10-27 10:07:52.704	HISTORICAL	2025-11-09 16:45:28.556

Рисунок 15 — Таблица inventory_transaction_audit

Реализовано три триггера:

Триггер 1 — Валидация остатков на складе:

Бизнес-правило: поднимать ошибку при попытке списать количество материала, недостаточного на складе.

Алгоритм:

- таблица — inventory_transaction;
- событие — BEFORE INSERT OR UPDATE;
- действия:
 1. Получить остаток материала.
 2. Проверить достаточность остатка.
 3. Вызвать ошибку, если недостаточно.

Триггер 2 — Аудит операций списания материалов:

Бизнес-правило: вести журнал всех операций списания материалов.

Алгоритм:

- таблица — inventory_transaction;
- событие — AFTER INSERT;
- действия:
 1. Создать запись в таблице inventory_transaction_audit.
 2. Сохранить данные операции (ID транзакции, материалы, сотрудник).
 3. Зафиксировать время и пользователя, выполнившего операцию.

Триггер 3 — Защита данных сотрудников:

Бизнес-правило: запретить удаление сотрудника, если у него есть активные заказы

или транзакции списания материалов.

Алгоритм:

- таблица — employee;
- событие — BEFORE DELETE;
- действия:
 1. Проверить наличие записей в material_order для удаляемого сотрудника.
 2. Проверить наличие записей в inventory_transaction.
 3. При наличии связанных данных отменить удаление с ошибкой.

Триггер 1 представлен в Листинге 1.

Листинг 1 — Триггер 1

```
-- Обновляем функцию триггера
CREATE OR REPLACE FUNCTION borgachev_tm.validate_inventory_balance()
RETURNS TRIGGER AS $$
DECLARE
    current_balance NUMERIC(10,2);
BEGIN
    -- Обработка разных типов операций
    IF TG_OP = 'INSERT' THEN
        -- Существующая логика для INSERT
        SELECT amount INTO current_balance
        FROM borgachev_tm.inventory
        WHERE inventory_id = NEW.inventory_id;

        IF current_balance IS NULL THEN
            RAISE EXCEPTION 'Материал с inventory_id=% не найден на складе', NEW.in-
inventory_id;
        ELSIF current_balance < NEW.amount_spent THEN
            RAISE EXCEPTION 'Недостаточно материала на складе. Остаток: %. Запрошено:
%',
                current_balance, NEW.amount_spent;
        END IF;
        RETURN NEW;

    ELSIF TG_OP = 'UPDATE' THEN
        -- Запрет на любые операции UPDATE
        RAISE EXCEPTION 'Обновление записей в таблице inventory_transaction запре-
щено. Используйте вставку новых записей для корректировок.';

    ELSE
        -- Для других операций
        RETURN NEW;
    END IF;
END;
$$ LANGUAGE plpgsql;

-- Обновляем триггер для поддержки INSERT и UPDATE
DROP TRIGGER IF EXISTS trg_validate_inventory_balance ON borgachev_tm.inven-
tory_transaction;
CREATE TRIGGER trg_validate_inventory_balance
BEFORE INSERT OR UPDATE ON borgachev_tm.inventory_transaction
FOR EACH ROW EXECUTE FUNCTION borgachev_tm.validate_inventory_balance();
```

Пример использования триггера представлен на Рисунках 16.

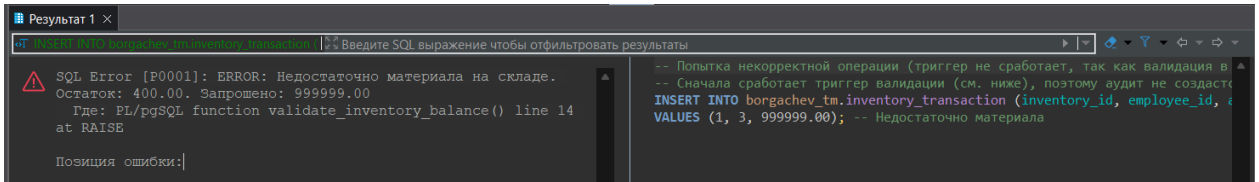


Рисунок 16 — Некорректная вставка

Триггер 2 представлен в Листинге 2.

Листинг 2 — Триггер 2

```
-- Создаем функцию аудита
CREATE OR REPLACE FUNCTION borgachev_tm.log_inventory_transaction()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO borgachev_tm.inventory_transaction_audit (
        transaction_id, inventory_id, employee_id, detail_id,
        amount_spent, transaction_date, operation
    )
    VALUES (
        NEW.transaction_id,
        NEW.inventory_id,
        NEW.employee_id,
        NEW.detail_id,
        NEW.amount_spent,
        NEW.transaction_date,
        'INSERT'
    );
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_audit_inventory_transaction
AFTER INSERT ON borgachev_tm.inventory_transaction
FOR EACH ROW EXECUTE FUNCTION borgachev_tm.log_inventory_transaction();

-- Успешное выполнение (корректное списание)
INSERT INTO borgachev_tm.inventory_transaction (inventory_id, employee_id,
amount_spent)
VALUES (1, 3, 10.00); -- Достаточно материала на складе
```

Пример использования триггера представлен на Рисунке 17.

id	transaction_id	inventory_id	employee_id	detail_id	amount_spent	transaction_date	operation	audit_timestamp	audit_user
1	1	1	3	4	50	2025-10-27 10:00:17.465	HISTORICAL	2025-11-09 16:45:28.556	system_migrati
2	2	2	3	1	30	2025-10-27 10:00:17.465	HISTORICAL	2025-11-09 16:45:28.556	system_migrati
3	3	5	6	2	25.5	2025-10-27 10:00:17.465	HISTORICAL	2025-11-09 16:45:28.556	system_migrati
4	4	7	3	[NULL]	10	2025-10-27 10:00:17.465	HISTORICAL	2025-11-09 16:45:28.556	system_migrati
5	5	1	[NULL]	[NULL]	100	2025-10-27 10:07:52.704	HISTORICAL	2025-11-09 16:45:28.556	system_migrati
6	9	1	3	[NULL]	10	2025-11-10 09:49:10.104	INSERT	2025-11-10 09:49:10.104	borgachev_tm

Рисунок 17 — Созданный аудит

Триггер 3 представлен в Листинге 3.

Листинг 3 — Триггер 3

```
CREATE OR REPLACE FUNCTION borgachev_tm.protect_employee_deletion()
RETURNS TRIGGER AS $$
DECLARE
    order_count INT;
    transaction_count INT;
BEGIN
    SELECT COUNT(*) INTO order_count
    FROM borgachev_tm.material_order
    WHERE employee_id = OLD.employee_id;

    SELECT COUNT(*) INTO transaction_count
    FROM borgachev_tm.inventory_transaction
    WHERE employee_id = OLD.employee_id;

    IF order_count > 0 OR transaction_count > 0 THEN
        RAISE EXCEPTION 'Нельзя удалить сотрудника ID %. Найдено связанных записей:
заказы - %, транзакции - %',
            OLD.employee_id, order_count, transaction_count;
    END IF;

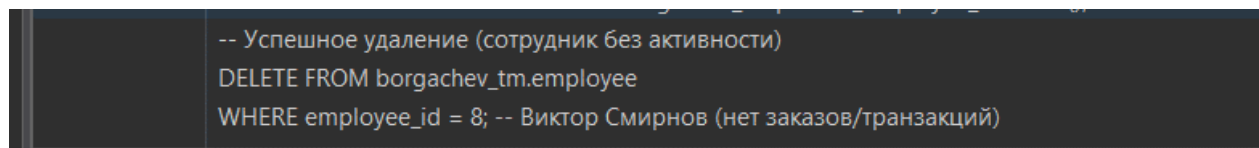
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_protect_employee_deletion
BEFORE DELETE ON borgachev_tm.employee
FOR EACH ROW EXECUTE FUNCTION borgachev_tm.protect_employee_deletion();

-- Успешное удаление (сотрудник без активности)
DELETE FROM borgachev_tm.employee
WHERE employee_id = 8; -- Виктор Смирнов (нет заказов/транзакций)

-- Ошибка при удалении активного сотрудника
DELETE FROM borgachev_tm.employee
WHERE employee_id = 3; -- Алексей Сидоров (есть заказы и транзакции)
```

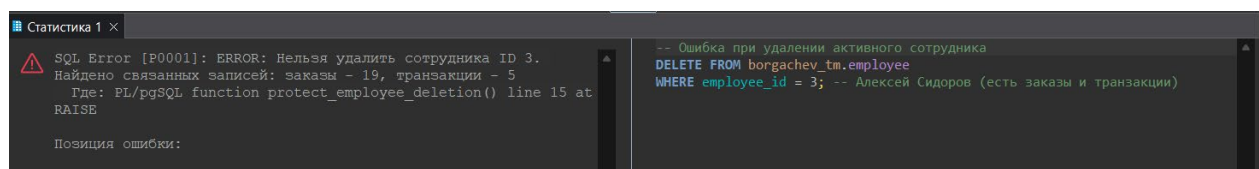
Пример использования триггера представлен на Рисунке 18.



```
-- Успешное удаление (сотрудник без активности)
DELETE FROM borgachev_tm.employee
WHERE employee_id = 8; -- Виктор Смирнов (нет заказов/транзакций)
```

Рисунок 18 — Проверка работы триггера – корректная

Пример некорректного использования триггера представлен на Рисунке 19.



```
SQL Error [P0001]: ERROR: Нельзя удалить сотрудника ID 3.
Найдено связанных записей: заказы - 19, транзакции - 5
Где: PL/pgSQL function protect_employee_deletion() line 15 at
RAISE

Позиция ошибки:

-- Ошибка при удалении активного сотрудника
DELETE FROM borgachev_tm.employee
WHERE employee_id = 3; -- Алексей Сидоров (есть заказы и транзакции)
```

Рисунок 19 — Некорректный вызов триггера

Задание №2: курсоры.

Скрипт 1: Явный курсор (анализ складских остатков).

Задача: рассчитать общий вес материалов на каждом складе в тоннах. При хранении материалов в килограммах.

Логика: используем явный курсор для построчной обработки данных о складах и

агрегации остатков.

Первый курсор представлен в Листинге 4.

Листинг 4 — Явный курсор

```
DO $$
DECLARE
    -- Объявление курсора для выборки складов и остатков
    warehouse_cursor CURSOR FOR
        SELECT
            w.warehouse_id,
            w.city,
            w.street,
            w.house_number,
            SUM(i.amount) AS total_kg
        FROM borgachev_tm.warehouse w
        JOIN borgachev_tm.inventory i ON w.warehouse_id = i.warehouse_id
        GROUP BY w.warehouse_id, w.city, w.street, w.house_number
        ORDER BY w.warehouse_id;

    warehouse_record RECORD;
    total_tons NUMERIC(10,2);
BEGIN
    -- Открытие курсора
    OPEN warehouse_cursor;

    RAISE NOTICE '=== СВОДКА ПО СКЛАДАМ ===';

    LOOP
        -- Извлечение данных
        FETCH warehouse_cursor INTO warehouse_record;
        EXIT WHEN NOT FOUND;

        -- Расчёт в тоннах
        total_tons := warehouse_record.total_kg / 1000;

        -- Вывод результатов
        RAISE NOTICE 'Склад #%: %, % % - Общий вес: % тонн',
            warehouse_record.warehouse_id,
            warehouse_record.city,
            warehouse_record.street,
            warehouse_record.house_number,
            total_tons;
    END LOOP;

    -- Закрытие курсора
    CLOSE warehouse_cursor;
END $$;
```

Пример использования представлен на Рисунке 19.

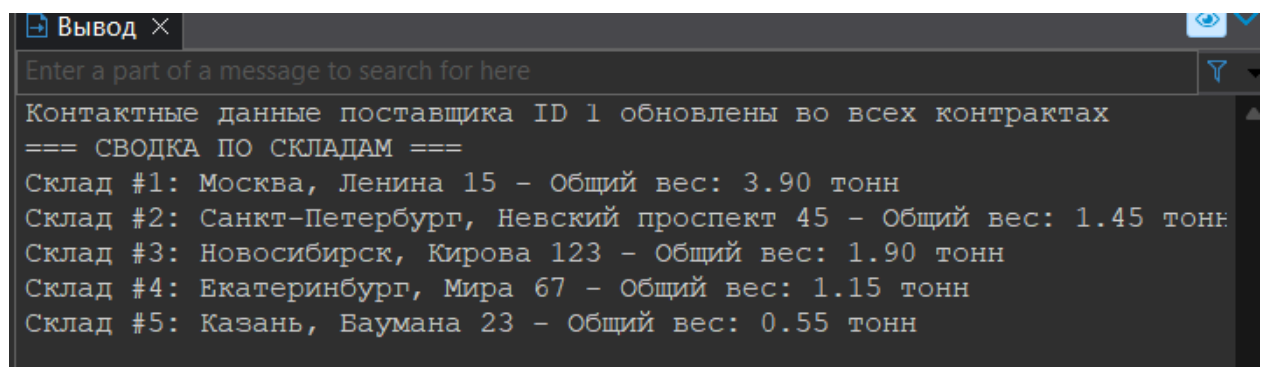


Рисунок 20 — Использование первого курсора

Скрипт 2: Неявный курсор (обновление стажа сотрудников).

Задача: автоматически обновить стаж работы (work_years) сотрудников на основе даты приёма (hire_date).

Логика: используем неявный курсор (цикл FOR) для пересчёта стажа для всех сотрудников.

Второй курсор представлен в Листинге 5.

Листинг 5 — Неявный курсор

```
DO $$
DECLARE
    employee_record RECORD;
    calculated_years INT;
BEGIN
    RAISE NOTICE 'ОБНОВЛЕНИЕ СТАЖА СОТРУДНИКОВ';

    -- Неявный курсор через цикл FOR
    FOR employee_record IN
        SELECT employee_id, hire_date
        FROM borgachev_tm.employee
    LOOP
        -- Расчёт стажа в годах
        calculated_years := EXTRACT(YEAR FROM AGE(CURRENT_DATE,
employee_record.hire_date));

        -- Обновление данных
        UPDATE borgachev_tm.employee
        SET work_years = calculated_years
        WHERE employee_id = employee_record.employee_id;

        RAISE NOTICE 'Сотрудник ID %: стаж обновлён до % лет',
            employee_record.employee_id, calculated_years;
    END LOOP;
END $$;
```

Пример использования представлен на Рисунке 20.

```
ОБНОВЛЕНИЕ СТАЖА СОТРУДНИКОВ
Сотрудник ID 1: стаж обновлён до 10 лет
Сотрудник ID 8: стаж обновлён до 2 лет
Сотрудник ID 3: стаж обновлён до 3 лет
Сотрудник ID 4: стаж обновлён до 1 лет
Сотрудник ID 6: стаж обновлён до 4 лет
Сотрудник ID 2: стаж обновлён до 5 лет
Сотрудник ID 5: стаж обновлён до 6 лет
Сотрудник ID 7: стаж обновлён до 2 лет
```

Рисунок 21 — Использование второго курсора